

Designing Larger Python Programs for Data Science

×

Module 1

^

Introduction to Larger Programs

✓

Course Introduction: Moving on to Larger Programs

Video

✓

Software Engineering vs. Data Analysis

Video

✓

Putting it All Together: Top Down Design

Reading

Report a problem with the course

Reading

✓

Random Story: Overview

Reading

Random Story: Planning

Video

Random Story Step 1

Programming Assignment

Random Story: from Parsing to Blank Types

Video

Random Story Step 2

Programming Assignment

Random Story: from Blank Types to Categories

Video

Random Story Step 3

Programming Assignment

Random Story: from Categories to Backreferences

Video

Random Story Finish

Programming Assignment

Module 2

▼

Monte Carlo Methods and Introduction to the Poker Project

Module 3

▼

Writing Test Cases and Identifying Sources of Error

Module 4

▼

Integrating Larger Programs

Random Story: Overview

In the next four lab assignments, you are going to be writing a program to generate a semi-random story, similar to the game "Mad Libs".

If you have never played Mad Libs, the way that it works is that one player has a story with blanks in it. Each blank contains a description of the kind of word that goes there (like "verb" or "place" or "food"). The player with the story asks the other players to name a word of that kind, and writes it in the blank. Once all blanks are filled in, the story is read aloud.

For example, you might have a story with blanks like this:

Once upon a time the king found a **_noun** under this throne. It was very **_adjective** but also **_adjective**. The queen was very **_emotion** when the king asked the Court Jester to **_verb** it, but everyone lived happily ever after.

In the game of Mad Libs, one person (let’s say Drew) looks at this story and asks the other person (let’s say Genevieve) to pick words that meet the descriptions in the blanks. Genevieve does not get to see the story, or know the context of the blanks. So for example, the interaction might go like this:

Drew: Pick a noun
Genevieve: flute
Drew: Pick an adjective
Genevieve: floppy
Drew: Pick another adjective
Genevieve: smelly
Drew: Pick an emotion:
Genevieve: thrilled
Drew: Pick a verb
Genevieve: eat

As Genevieve picks these words, Drew writes them into the blanks to make the final story:

Once upon a time the king found a **flute** under this throne. It was very **floppy** but also **smelly**. The queen was very **thrilled** when the king asked the Court Jester to **eat** it, but everyone lived happily ever after.

Your random story program will follow the same principles, except for two differences. The first is that instead of interactively picking words, it will have input files for each category. For the above example, there would be one file with nouns, one file with adjectives, one file with emotions, and one file with verbs. Your program would then pick randomly from the appropriate category’s file.

The other is that we will have *backreferences*, where our story can specify the reuse of a word that was previously selected. The above story says “it” a lot as we want to refer back to our noun. However, if we just had another **_noun** blank, we would pick a random noun which might be different. Instead, we will use **_0** to say to reuse the contents of the first blank, **_1** for the second, and so on. So we might change our story to:

Once upon a time the king found a **_noun** under this throne. The **_0** was very **_adjective** but also **_adjective**. The queen was very **_emotion** when the king asked the Court Jester to **_verb** the **_2, _3 _0**, but everyone lived happily ever after.

Now with the same selections as above, we would have the story:

Once upon a time the king found a **flute** under this throne. The flute was very **floppy** but also **smelly**. The queen was very **thrilled** when the king asked the Court Jester to **eat** the **floppy, smelly flute**, but everyone lived happily ever after.

This assignment involves several skills you have already seen in prior courses, such as iteration, working with strings, reading from files, and using dictionaries. However, this program is a bit larger and more complex than what you have done before. The learning objective here is to see how to break a larger program down into smaller pieces—making something that does part of the task first, then refining it until you get what you need. Such an approach lets you test your code as you go, so that it is much easier to debug.

We will discuss the specifics of what to do in each step in the READMEs for each lab assignment.

Go to next item

✓ Completed

Like

Dislike

Report an issue